

Vulnerability Scanner Process Flows

Laptop OpenVAS → central findings → gap reports and exports

Complete startup, feed, authentication, per-IP scan, coverage, scoring and report diagrams.

Includes separately configured central engines and their actual implementation limits.

Scanner source 1.5.3-v45.7. Live Windows/Docker/Greenbone acceptance remains pending.

Versioned scope for all 52 guide families; actual service evidence and credential/network gaps are reported.

SOURCE REVIEW + SCANNER RELEASE 1.5.3-v45.7

06 October 2026 | Separate execution paths, shared evidence rules

Contents and diagram key

Scanner / System boundary and complete execution	3
Laptop / Startup and container boot	4
Laptop / Greenbone feeds and scan readiness	5
Laptop / Central discovery, TLS and token binding	6
Central / Registration, readiness and launch	7
Laptop / Job claim, resume and ownership	8
Laptop / Adaptive per-IP task scheduling	9
Laptop / Port profiles and discovery observations	10
Laptop / Create and start the OpenVAS task	11
Service policy / Request to actual target scope	12
Service evidence / Native report to family status	13
Service scope / Guide families and remaining validation	14
Laptop / Polling, task completion and cancellation	15
Laptop / Report pagination and native evidence	16
Central / Completion and coverage verdict	17
Central / Durable per-IP progress to the UI	18
Central / Normalize, deduplicate and retain findings	19
Scoring / Technical severity and contextual priority	20
Central optional / Nmap and Nuclei execution	21
Central optional / ZAP, Trivy and Wazuh routes	22
Central optional / Licensed Nessus and native imports	23
Scanner report / Coverage review to gap assessment	24
Monitoring / Performance without false completion	25
Release 1.5.3 / Changes and verification	26
Deployment / Apply the Laptop and central fixes	27
Implementation map and primary references	28

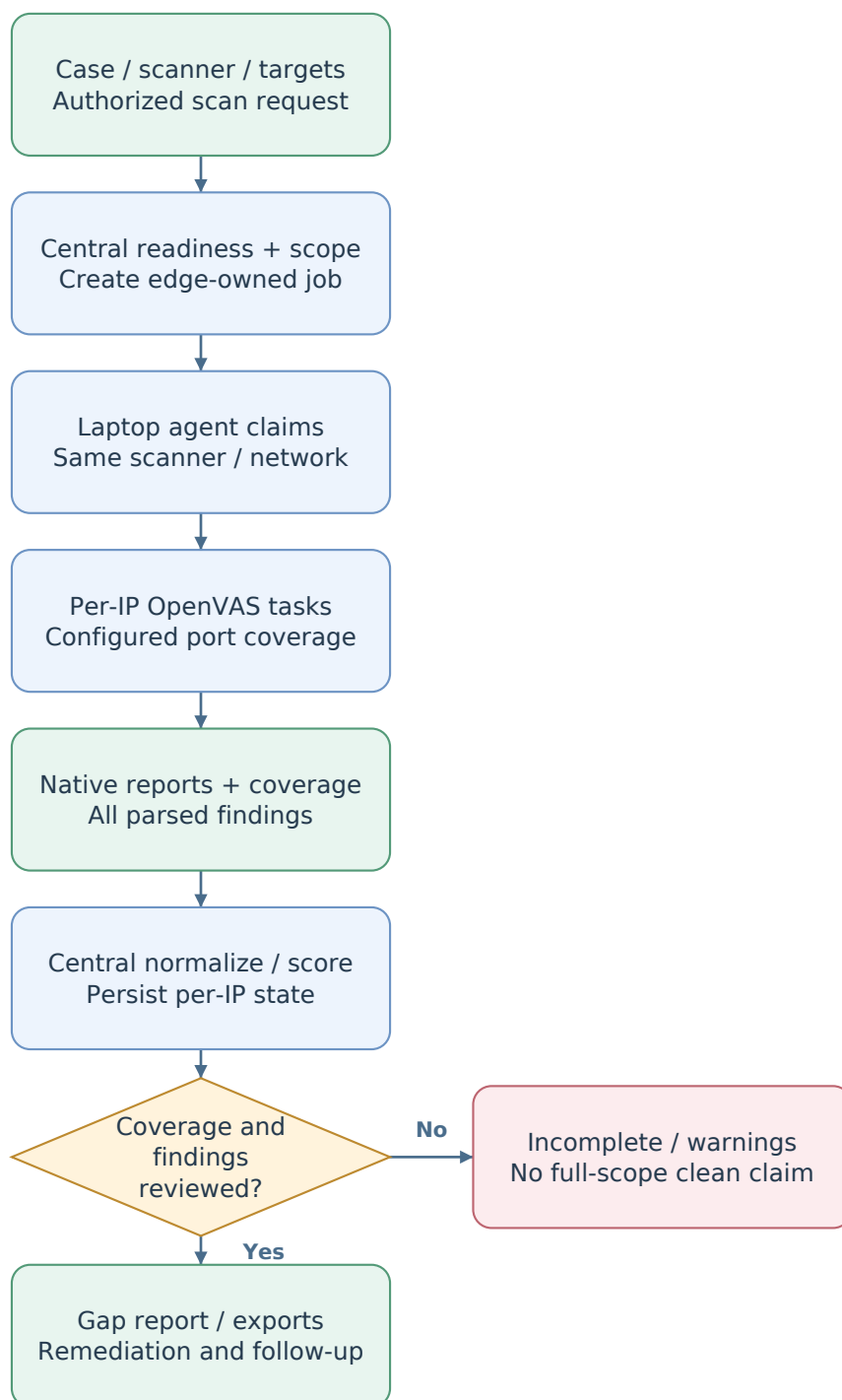
Diagram key

 Process	 Gate / branch	 Retained evidence
 Key / crypto	 Gap / failure	 External workflow

Solid arrows show implemented transitions. Dashed arrows show an explicitly external or conditional path. A gap is evidence of a limitation; it does not establish that the requested data never existed.

Scanner / System boundary and complete execution

The attached Laptop Scanner runs local Greenbone/OpenVAS. The central Vulnerability service owns case scope, job dispatch, persisted findings and reports.



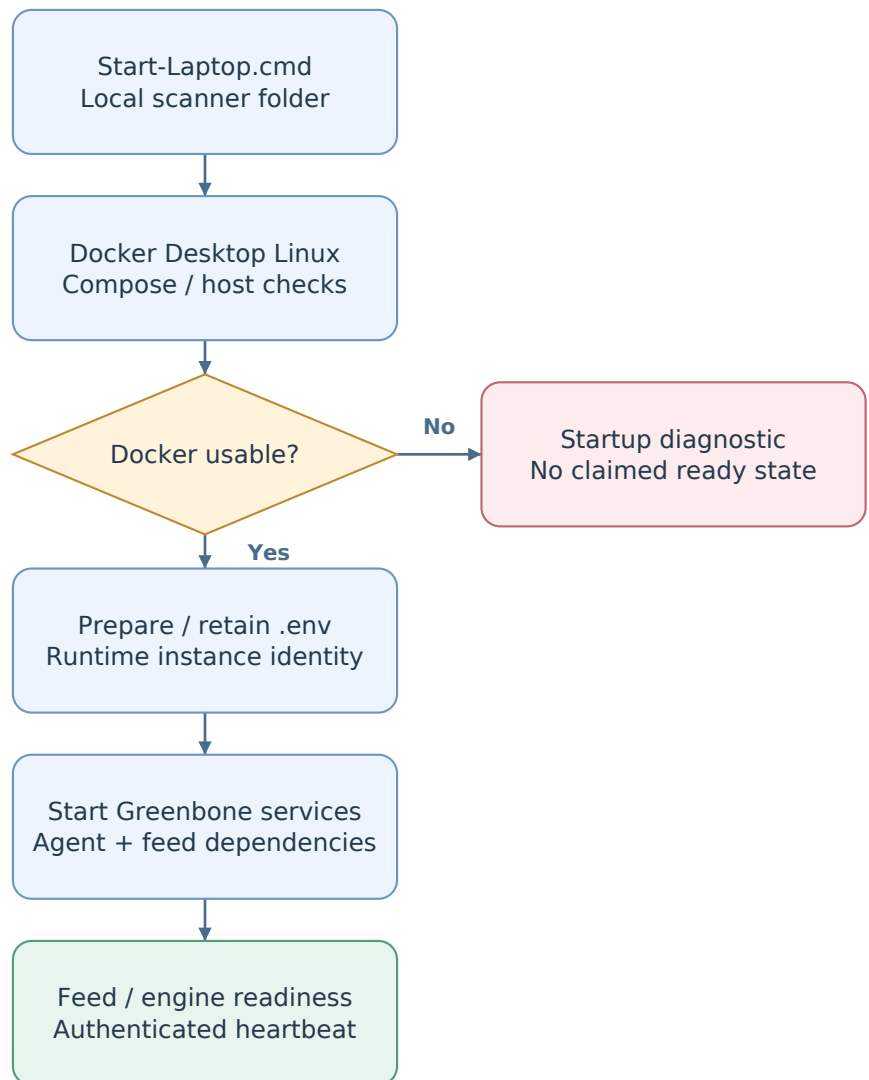
Other engines Nmap, Nuclei, ZAP, Wazuh, Trivy and licensed Nessus are separately configured central adapters. They are not all bundled into the portable Laptop Scanner.

Assessment semantics Task completion, useful findings and proven assessment coverage are separate. A zero-finding or skipped stage does not establish that the requested hosts are clean.

Implementation: laptop-scanner/scanner-agent/agent/main.py; backend/app/services/scan_orchestrator.py; multi_scanner_clients.py

Laptop / Startup and container boot

Start-Laptop.cmd runs the laptop startup script. An existing installation keeps its environment, instance identity and Greenbone data volumes.



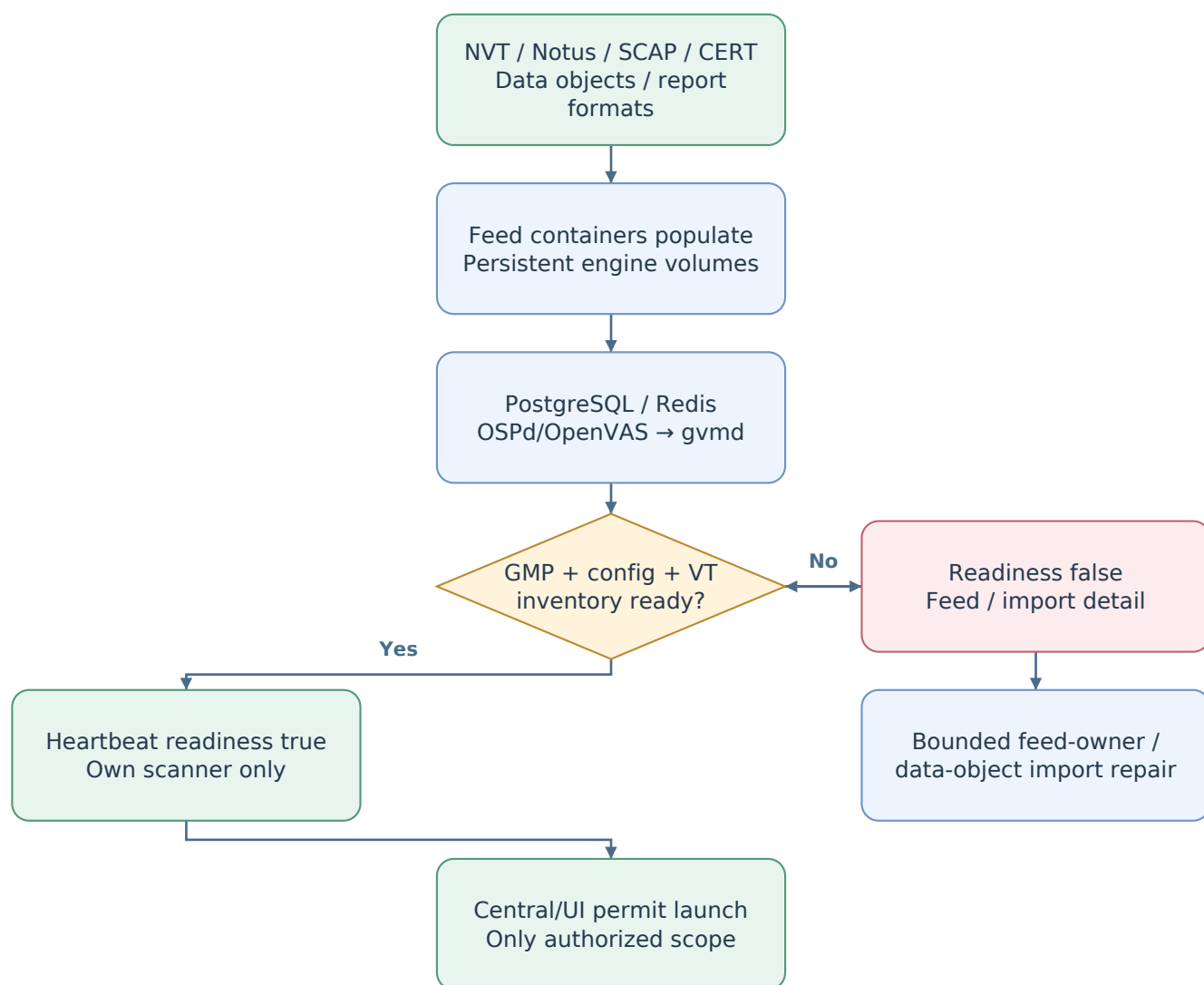
Secret handling Use placeholder examples for a fresh deployment. Retain the user's real `.env` and runtime tokens locally; release ZIPs exclude those files.

Update path `Apply-Scanner-v1.5.3.ps1` rebuilds the agent/bootstrap and recreates only the agent. It does not delete GVM database/feed volumes or restart the engine solely to update agent code.

Implementation: `laptop-scanner/Start-Laptop.cmd`; `scripts/start-laptop.ps1`; `Apply-Scanner-v1.5.3.ps1`

Laptop / Greenbone feeds and scan readiness

A GMP socket alone is insufficient. The agent waits for the scan configuration and usable VT inventory before accepting new work.



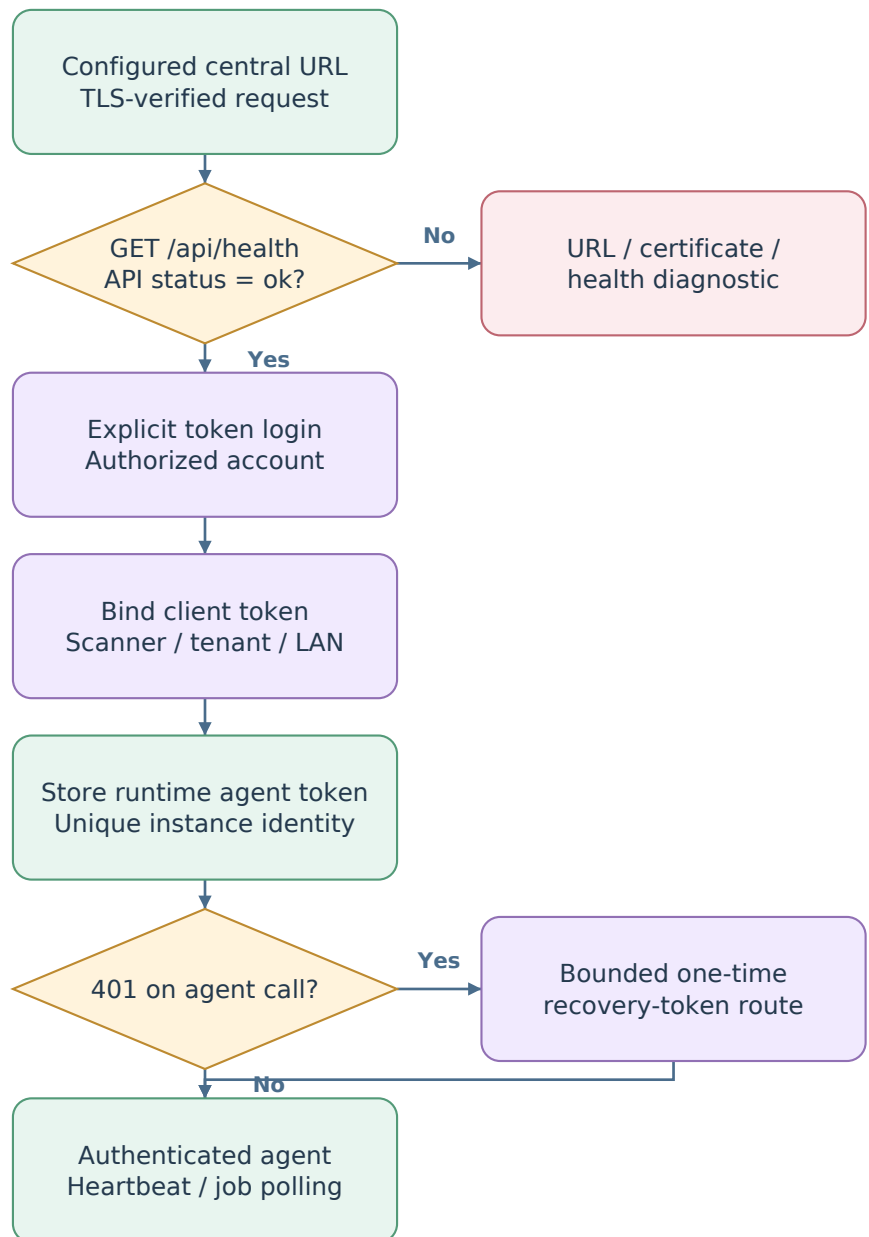
Readiness threshold The current checks require the expected Full-and-fast configuration and a usable NVT inventory (10,000+), not merely a listening socket.

Bounded startup Feed import repair is an existing startup process step, not a separate persistent healing agent. Failures remain visible; the code does not erase feed volumes to force a green state.

Implementation: laptop-scanner/scanner-agent/check_greenbone_ready.py; check_greenbone_feed_state.py; scripts/start-laptop.ps1

Laptop / Central discovery, TLS and token binding

Bootstrap verifies the central API, obtains the requested account token and binds the laptop scanner identity.



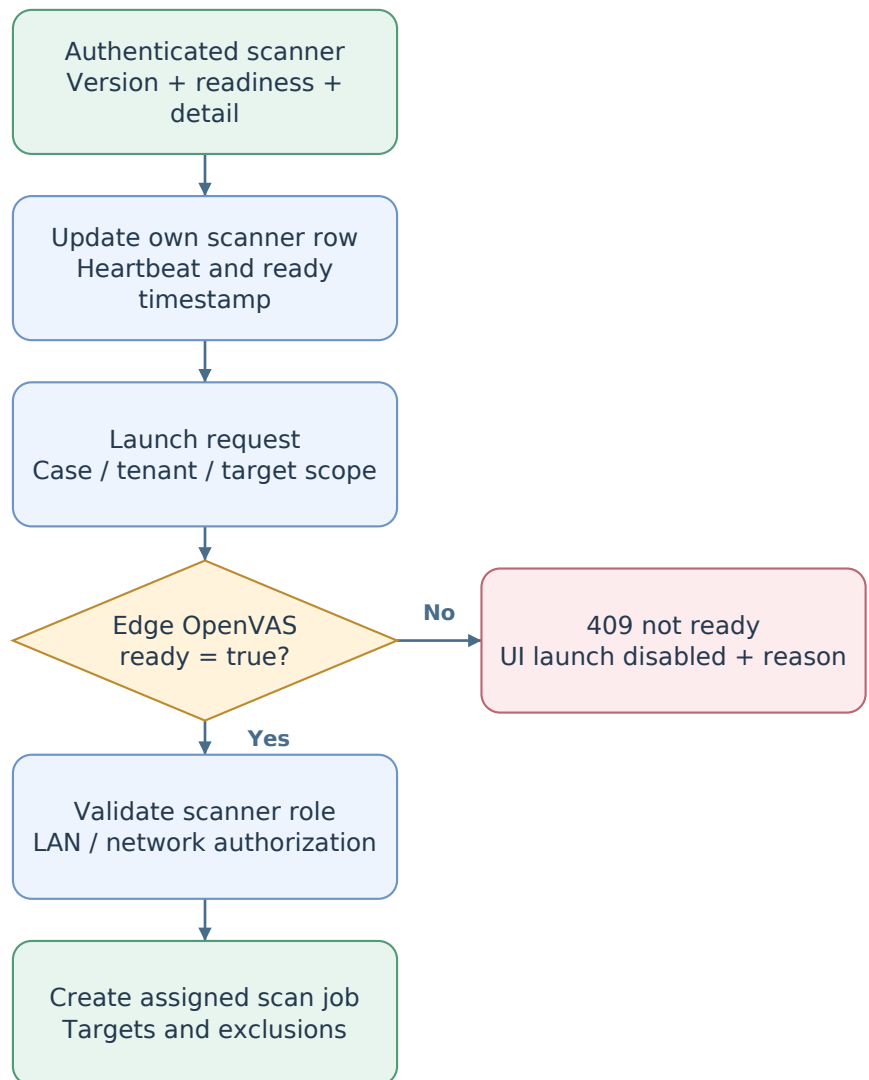
URL contract Redirect canonical nonsecret GET requests when needed. Validate certificate hostname/SNI and API-shaped health; a generic {ok:true} response is not accepted as this API.

Security / ownership Tokens and passwords do not enter logs or packaged runtime directories. Scanner instance identity fences active tasks and avoids competing laptops posting into the same owned job.

Implementation: laptop-scanner/scanner-agent/agent/bootstrap_control_plane.py;
backend/app/services/scanner_agent_auth.py

Central / Registration, readiness and launch

Authenticated heartbeats now update only the calling scanner. Launch and claim decisions read that scanner's real engine readiness.



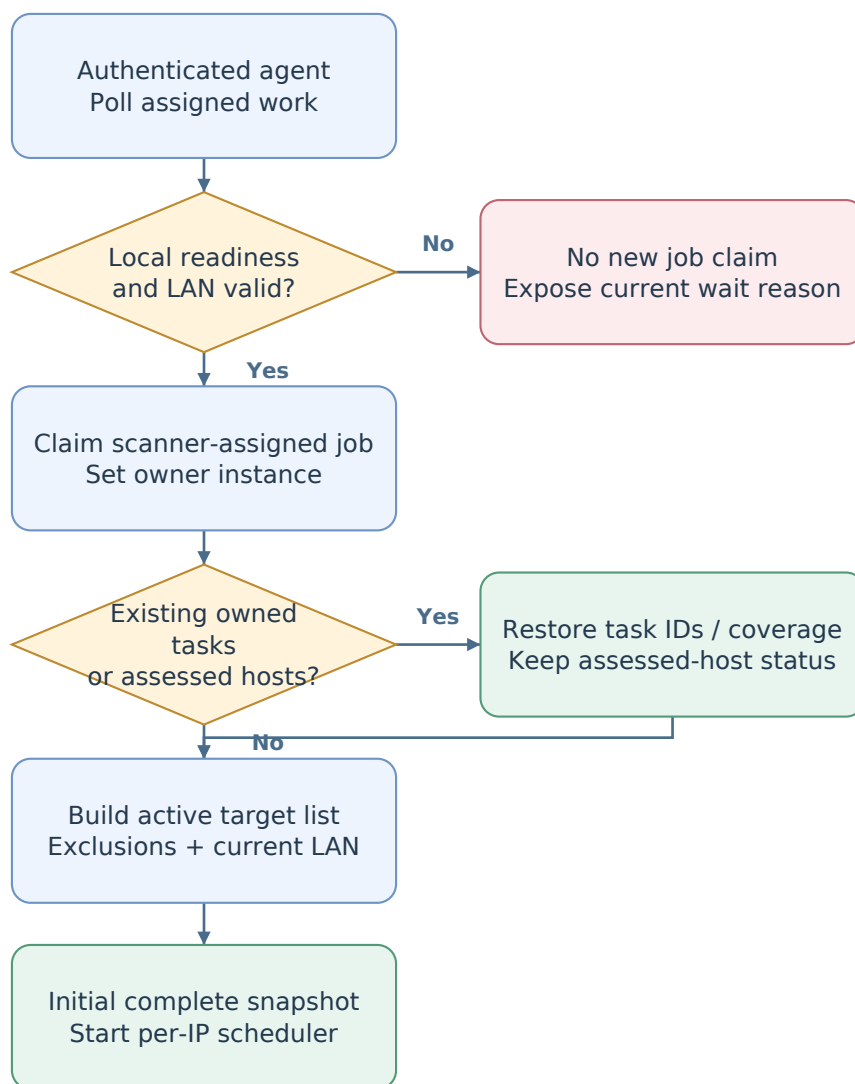
Fixed behavior One laptop heartbeat can no longer mark other active edge scanners online. The API exposes `openvas_ready`, status detail and readiness time to the UI.

Schema Migration `037_firm_vuln_readiness.sql` adds readiness columns. The authenticated heartbeat has an idempotent compatibility check for an existing deployment.

Implementation: `backend/app/routers/scanner_agent.py`; `routers/vuln.py`; `services/scanner_agent_jobs.py`; `db/sql_migrations/037_firm_vuln_readiness.sql`

Laptop / Job claim, resume and ownership

Default laptop execution accepts one central job at a time. Multiple per-IP tasks run inside that job.



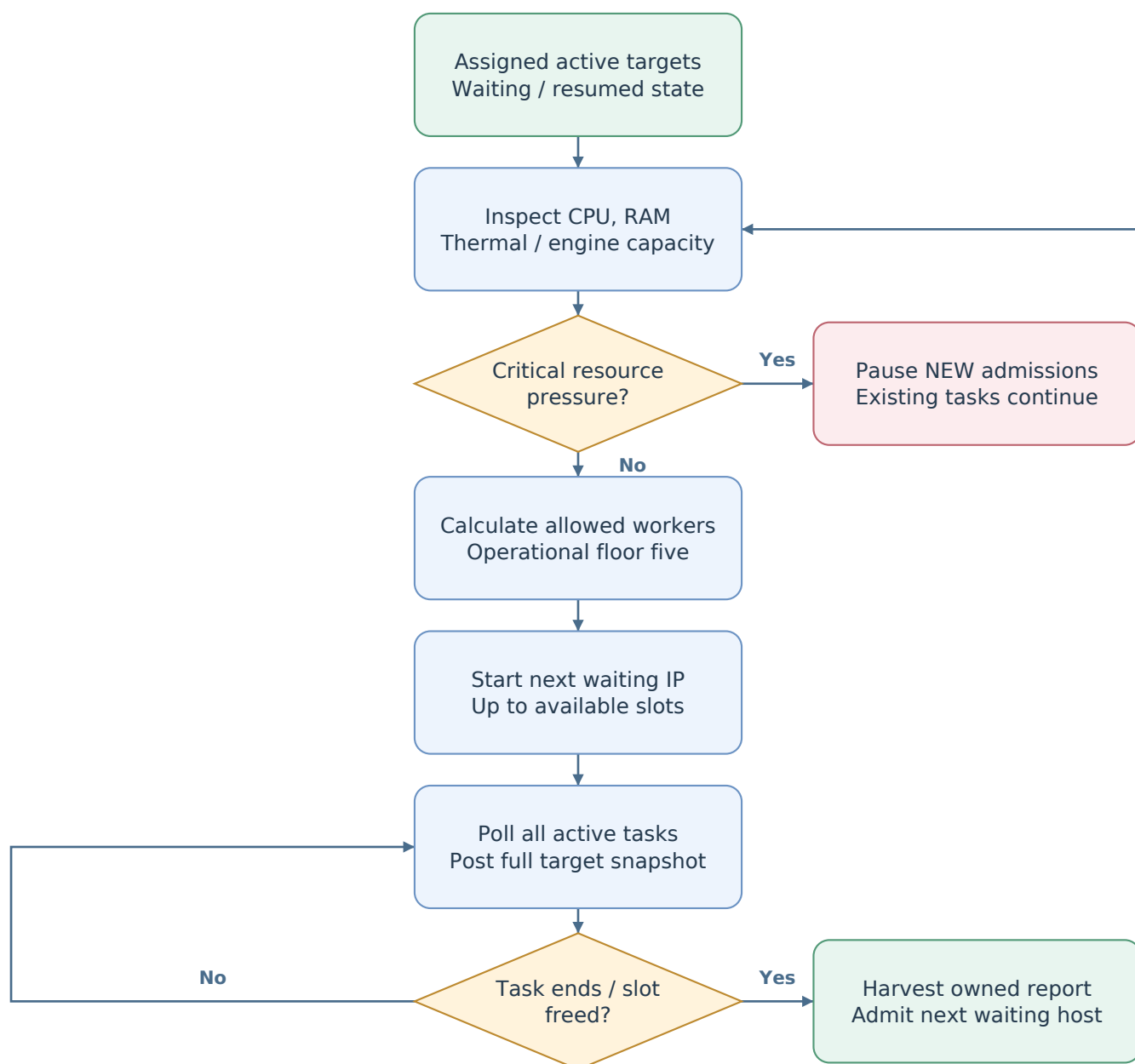
Resume precision Persisted task IDs allow a restarted agent to observe its tasks. Previously assessed hosts are shown complete/incomplete according to their retained coverage, rather than incorrectly reset to waiting.

Scope change A changed network stops only the old job's owned tasks. Results from different LAN scopes must not merge into one assessment.

Implementation: laptop-scanner/scanner-agent/agent/main.py; agent/target_progress.py; backend/app/services/scanner_agent_jobs.py

Laptop / Adaptive per-IP task scheduling

One OpenVAS task is created per IP. The scheduler fills available slots and retains waiting hosts in every progress snapshot.



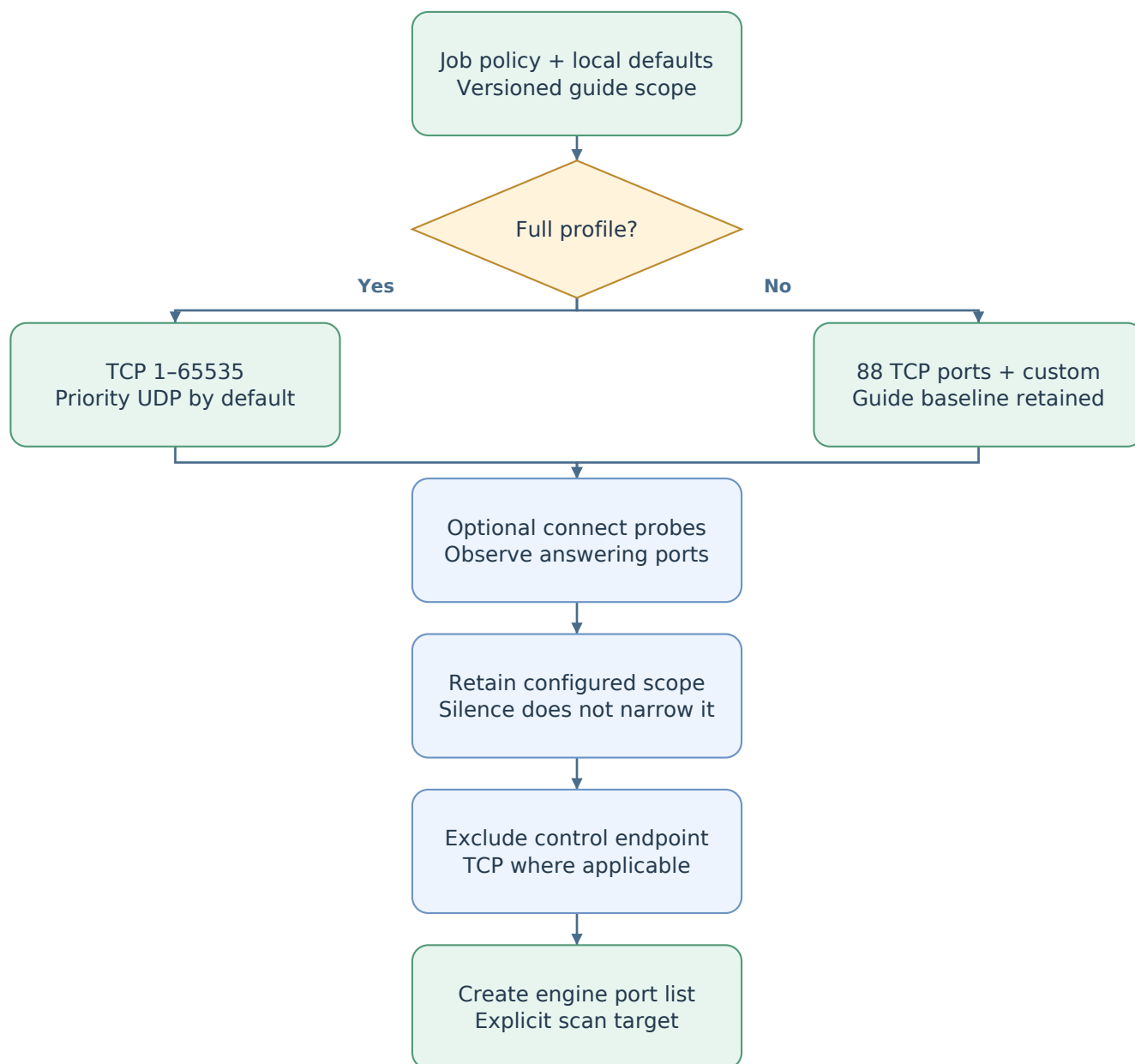
Concurrency Auto mode commonly uses five to six per-IP workers within the configured maximum. The example and Compose maximum may differ; inspect effective environment. Resource pressure pauses starts instead of killing unfinished scans.

Throughput estimate Six workers at an observed 20-minute mean would imply 18 IPs/hour. This is planning arithmetic, not a live measured guarantee. Full port sweeps, credentials, feeds and host behavior determine actual duration.

Implementation: laptop-scanner/scanner-agent/agent/capacity.py; agent/main.py

Laptop / Port profiles and discovery observations

Full defaults to TCP 1-65535 and 30 priority UDP ports. Named Fast/Common/Vuln modes retain 88 TCP ports, the guide baseline, selected UDP mode and custom additions.



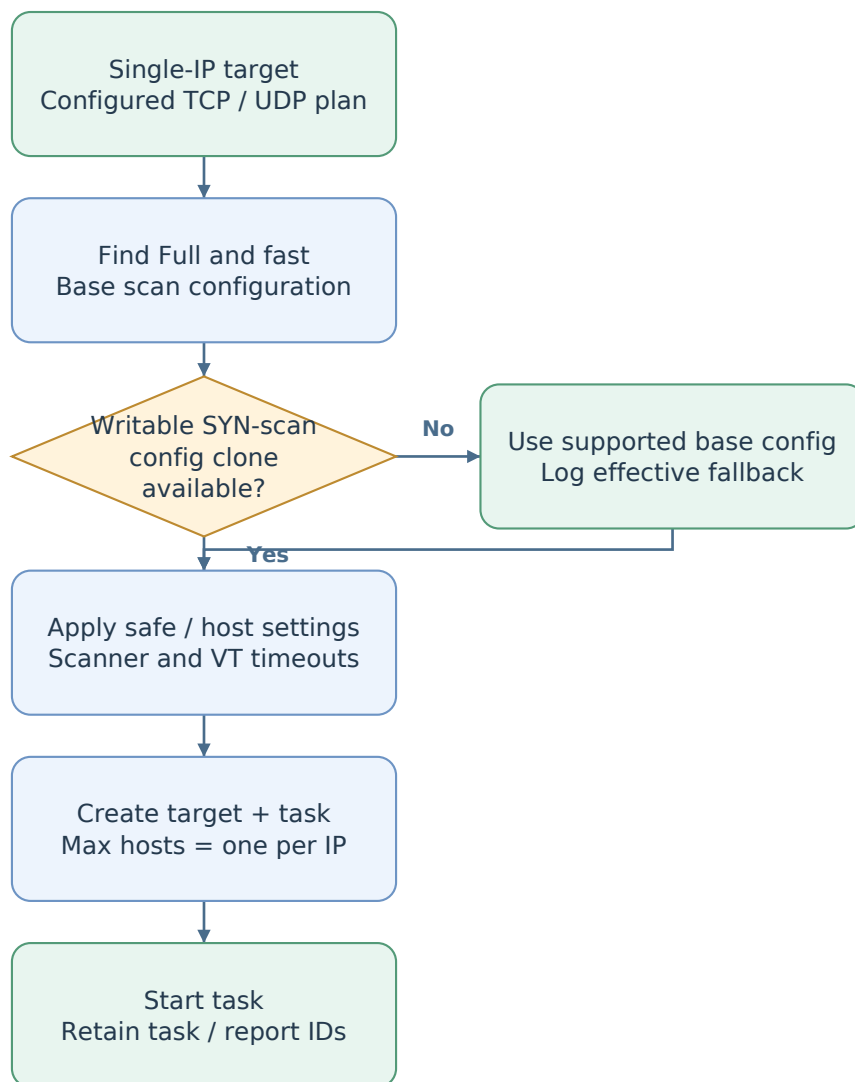
Coverage fix Fast discovery is observational. A probe that sees only SMB, or sees nothing, cannot remove SSH/TLS and other configured services from the OpenVAS target.

UDP Priority UDP includes the guide's 17-port baseline, six additional transports and seven existing infrastructure ports. UDP off/full applies to every named TCP mode. Effective exclusions/counts are recorded.

Implementation: agent/service_policy.py; service_catalog.json; gmp_local.py; port_discovery.py; vuln_ports.json

Laptop / Create and start the OpenVAS task

Task construction records the exact port plan and configuration before starting the engine.



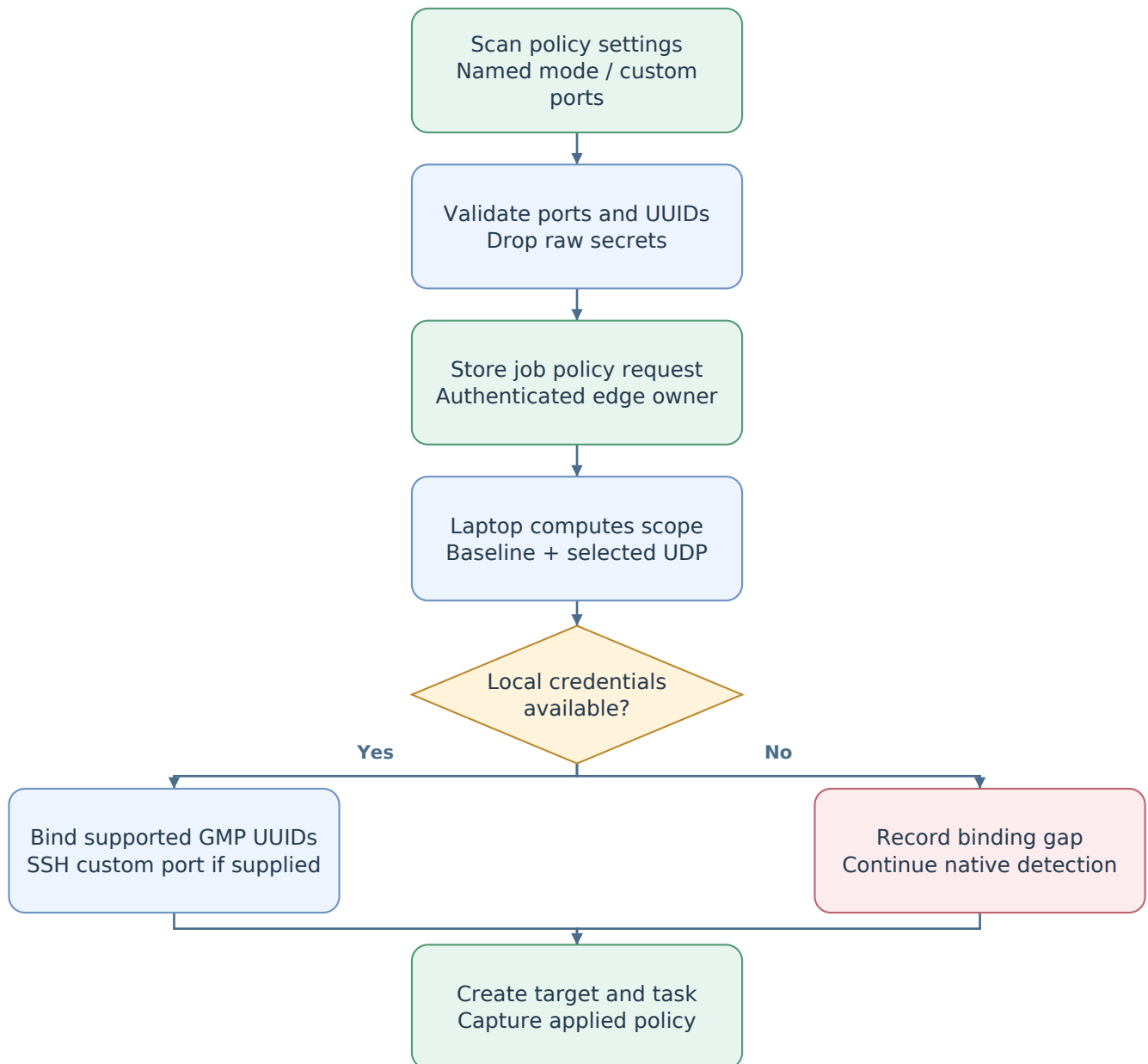
Quality defaults Full profile uses `optimize_test=no`; VT timeout 320 s and scanner-plugin timeout 36,000 s avoid premature port-scan truncation. Safe checks and service recognition remain enabled.

Compatibility Record effective target range, config/scanner IDs, feed version, applied preferences and SHA-256 at task creation. Explicit local GMP UUIDs bind SSH/SMB/SNMP/ESXi credentials; unsuccessful binding leaves a visible gap and detection continues.

Implementation: `laptop-scanner/scanner-agent/agent/gmp_local.py` (`_ensure_portscan_config`, `LocalOpenVAS`)

Service policy / Request to actual target scope

A policy is snapshotted when the central job is created, so later policy edits cannot change the claimed job's request.

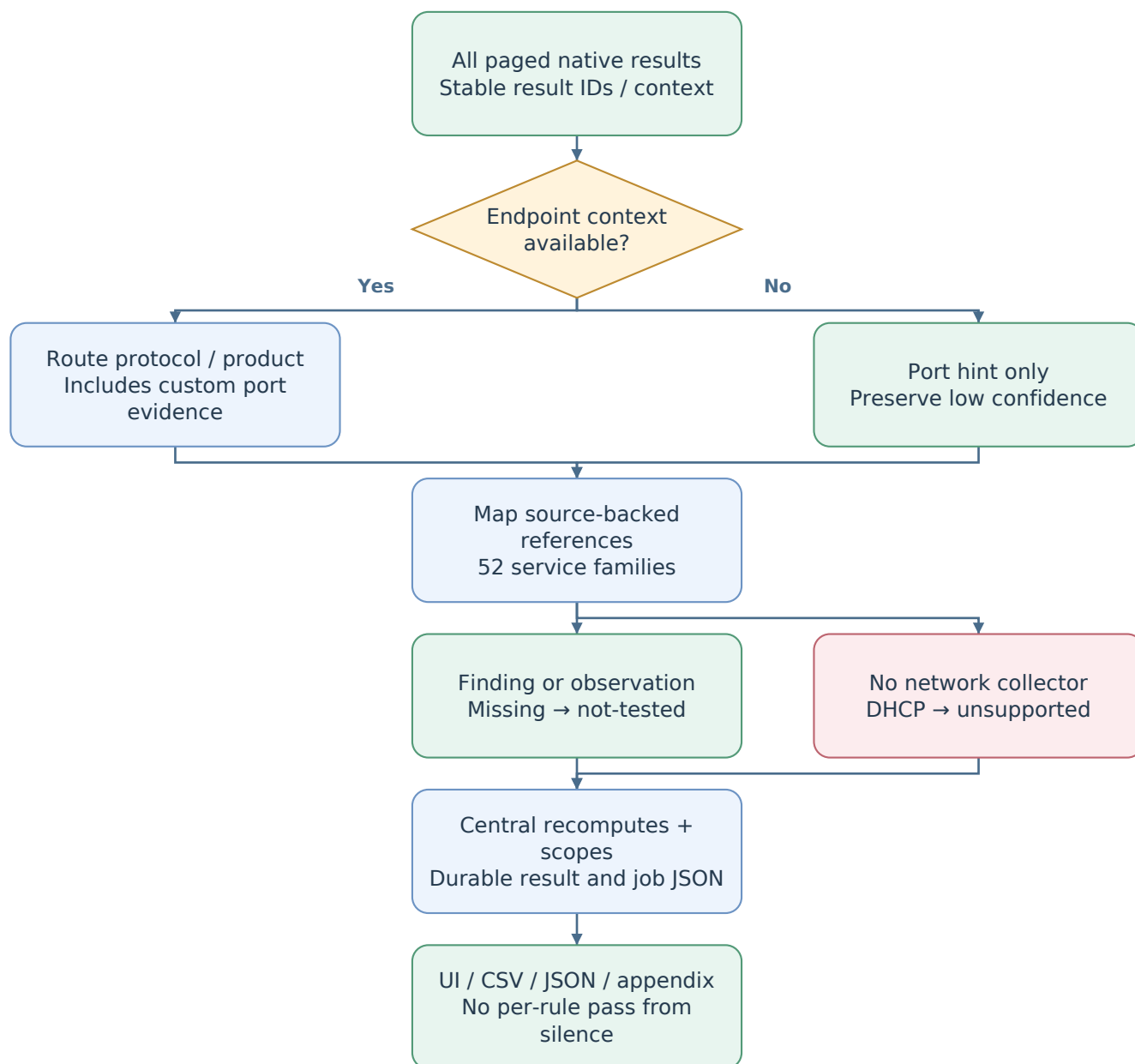


Credential boundary Credentials must already exist in the local Greenbone vault. A central credential_ref is not a local GMP UUID. WMI/WinRM and product-specific authenticated inventory depend on supported native feed methods; no new universal collector is implied.

Implementation: backend/app/routers/vuln.py; scanner_agent_jobs.py; agent/service_policy.py; gmp_local.py

Service evidence / Native report to family status

A service label is a source-backed observation. Port numbers alone do not identify a product or establish vulnerability.



Evidence limit Native plugin-name references can associate a family without proving a precise fingerprint; they remain references, not a fabricated version/CPE. Missing score is explicitly marked. Configuration IP forwarding requires source-backed role/configuration evidence.

Implementation: agent/service_policy.py; gmp_local.py; backend/app/services/scanner_agent_jobs.py; vuln_report_export.py

Service scope / Guide families and remaining validation

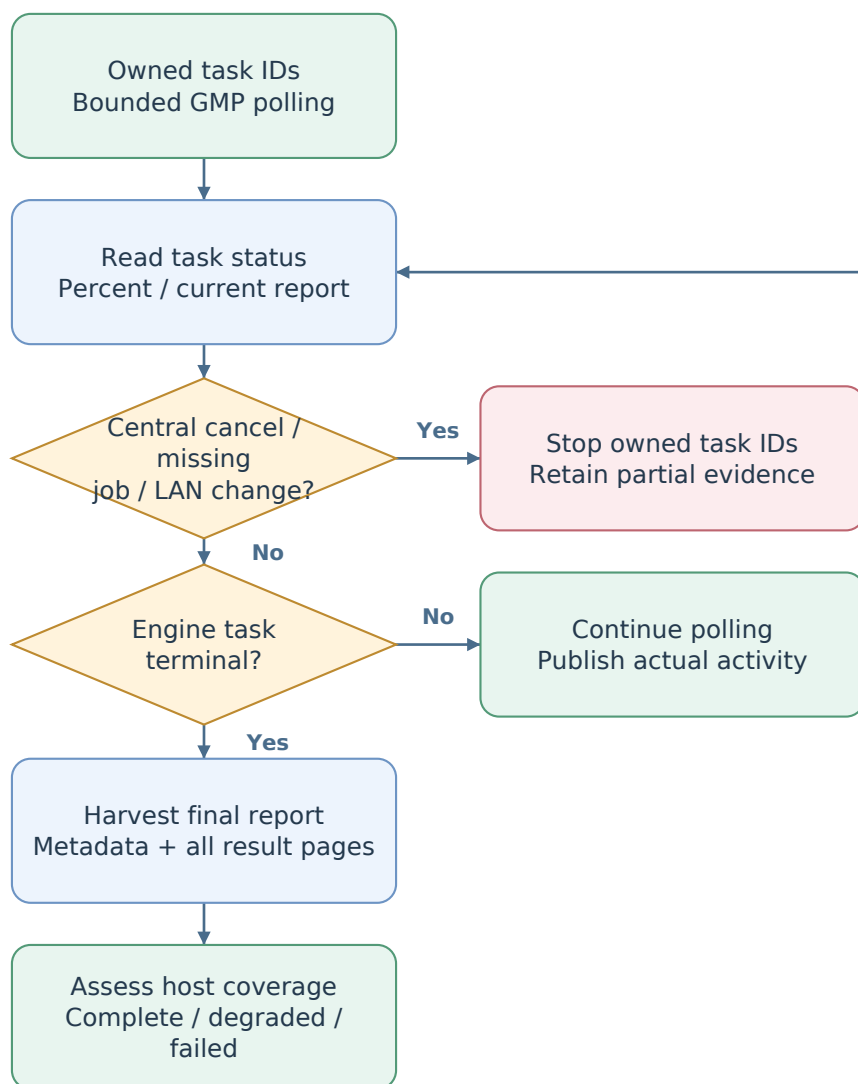
The attached guide is implemented as a versioned registry and native-engine routing contract. It is not evidence that every product check was exercised on a live target.

Group	Registered scope / current boundary
Windows / identity	SMB/CIFS, RPC, WMI, WinRM, RDP, LDAP/LDAPS, Kerberos, AD Global Catalog. Local credential binding is supported; successful authentication/privilege needs native evidence.
Remote / mail / transport	SSH OpenSSH/Dropbear, FTP/FTPS, SFTP, HTTP, HTTPS/TLS, SMTP/SMTPS, POP3/POP3S, IMAP/IMAPS, DNS, SNMP, Telnet, TFTP. Legacy TLS / STARTTLS tests remain native-feed and runtime dependent.
Data / platform	MSSQL, MySQL/MariaDB, PostgreSQL, Oracle, MongoDB, Redis, Elasticsearch, VNC, VMware, Docker, Kubernetes, kubelet, Jenkins, GitLab/Git, RabbitMQ, Kafka, Memcached.
Application servers	Tomcat, JBoss/WildFly, IIS, Apache HTTPD, nginx, WebLogic, WebSphere. Custom ports and actual product identity are retained; titles do not set severity.
Network / printer / RPC	IPsec/IKE, NTP, DHCP, IPP, JetDirect, NFS and rpcbind. DHCP/SSDP/mDNS network collectors and a new authenticated IPv4/IPv6 collector are not implemented by the per-IP agent.
Vendor appliances	Cisco, Fortinet, Palo Alto, Juniper, Aruba, HP/HPE, Dell, MikroTik, VMware and TP-Link are retained as intended vendor scope; rule coverage depends on native fingerprint/feed evidence.
Detailed list	Aetheris-Service-Coverage-List.csv and the updated Service Implementation Guide list every one of the 52 entries, hint ports, requested checks and remaining acceptance.

Implementation: agent/service_catalog.json; Aetheris-Scanner-Service-Implementation-Guide-Updated.pdf

Laptop / Polling, task completion and cancellation

Task status, coverage and report evidence control completion. A high percent alone is not a completed full scan.



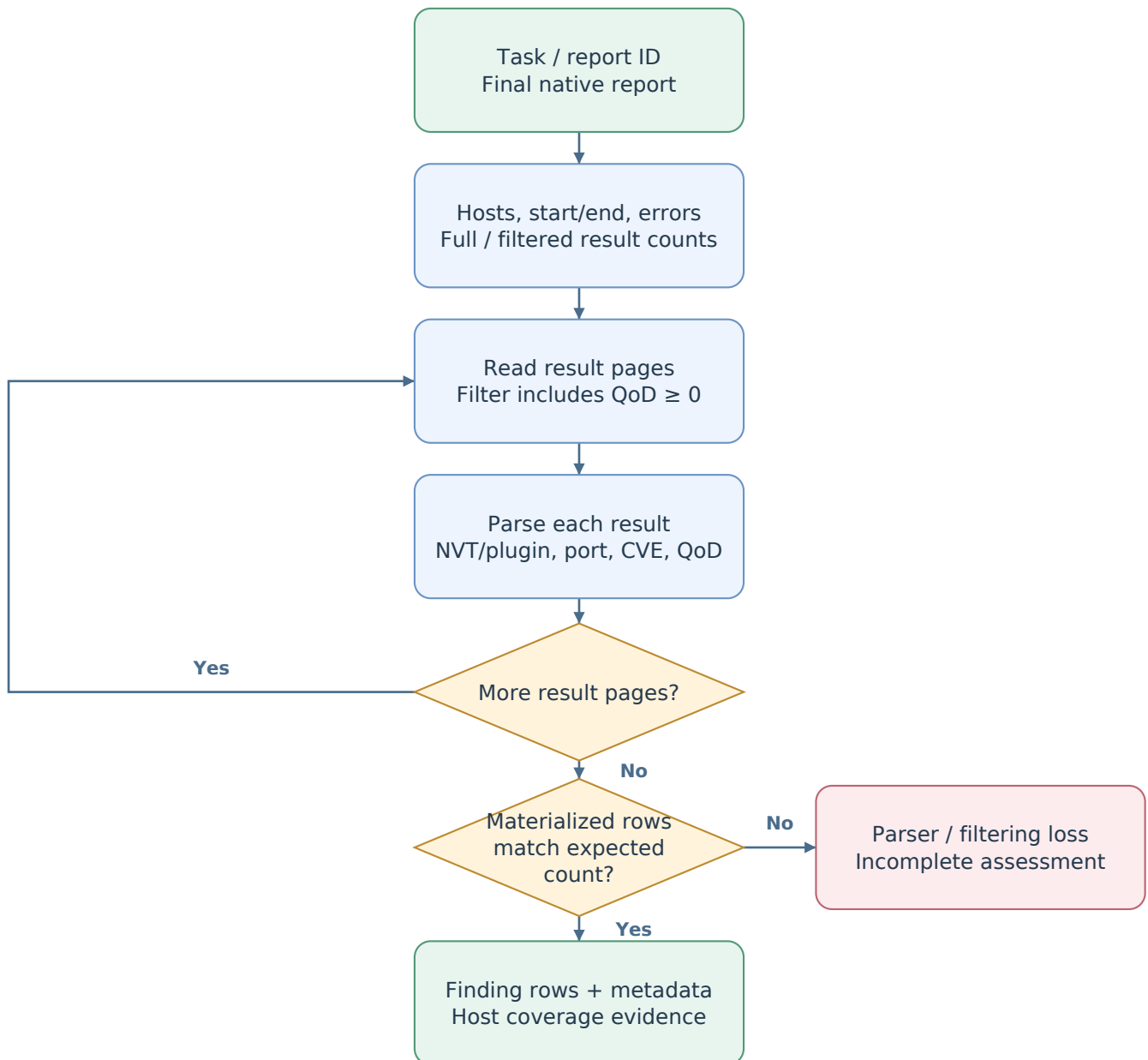
No forced tail harvest HIGH_PROGRESS_STALL_SEC=0 disables degraded high-percent harvesting by default. An unfinished port scanner is allowed to finish; optional limits must retain incomplete coverage.

Timeouts The agent uses bounded calls and optional explicit runtime ceilings. It does not abort a valid scan simply because an IP's percentage did not move for one minute.

Implementation: laptop-scanner/scanner-agent/agent/main.py; agent/gmp_local.py

Laptop / Report pagination and native evidence

Read Greenbone XML without silently dropping low-QoD or informational results. Keep each source finding's identity.



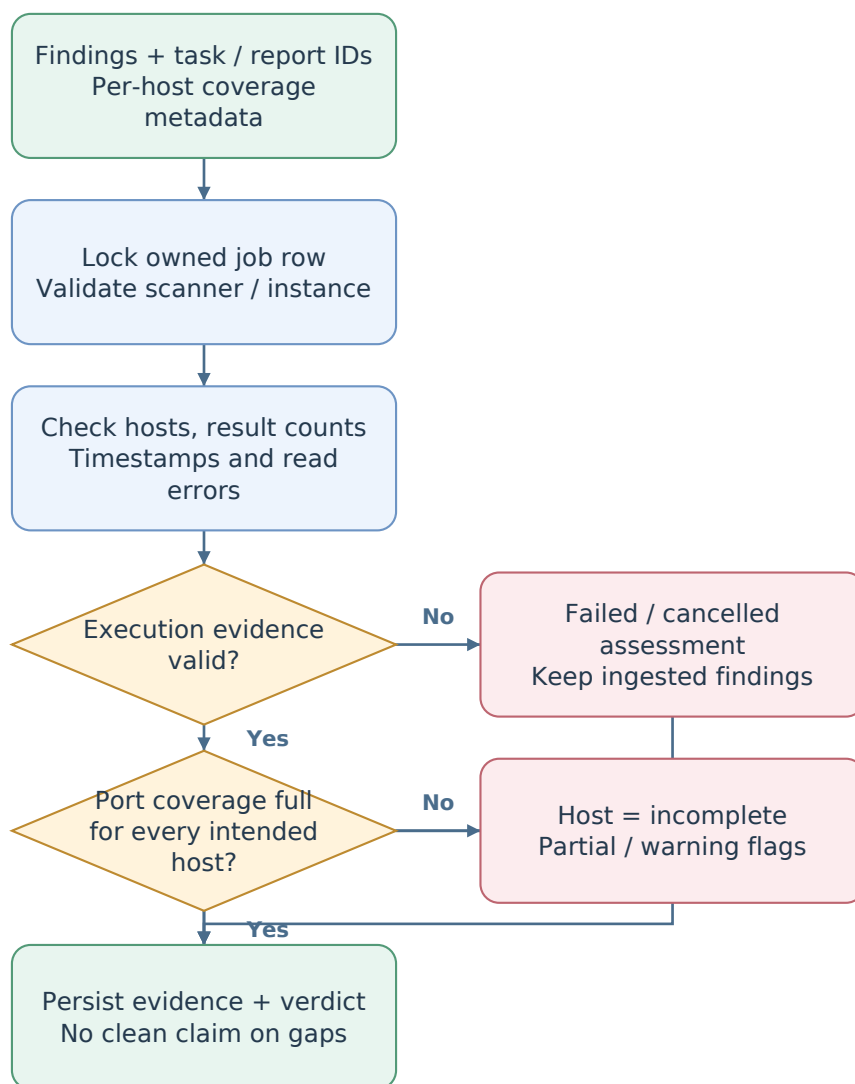
Retained details Keep task/report IDs, source result ID, plugin/NVT identifiers, all available CVSS version signals, QoD, references, CVEs/CWEs, host, protocol and port.

Integrity Compare the report's declared count with uploaded parsed rows. A report with 81 results and only three uploaded rows cannot become a valid assessment.

Implementation: `laptop-scanner/scanner-agent/agent/gmp_local.py (_report_evidence_from_xml, _vulnerabilities_from_report)`

Central / Completion and coverage verdict

Result ingestion verifies execution evidence and preserves degraded host coverage even when the top-level task says Done.



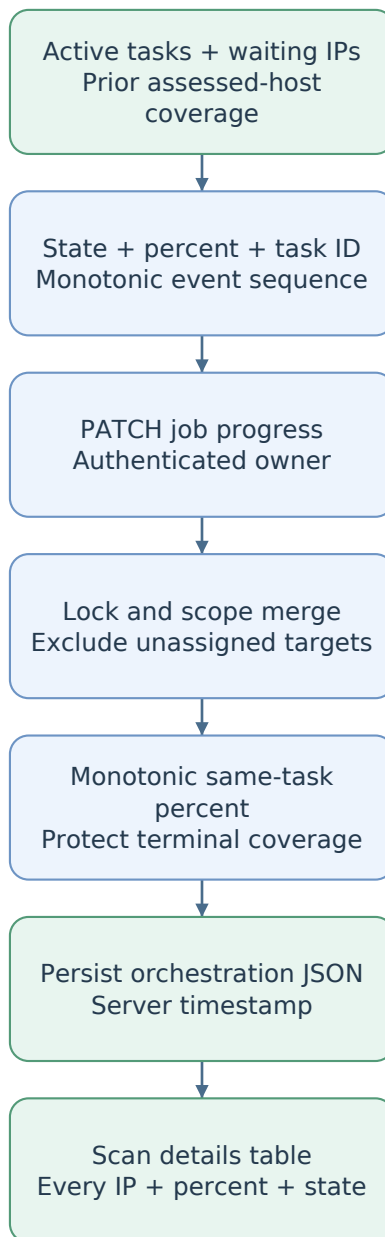
Three consistent views Stored scan result evidence, orchestration state and the result API response now agree on partial, clean_eligible and warnings. A degraded per-IP scan stays incomplete.

Clean eligibility Task execution can finish with warnings. Missing requested hosts, parser count mismatch, skipped scope or a truncated port scanner prevent a full-scope clean conclusion.

Implementation: backend/app/services/scanner_agent_jobs.py;
laptop-scanner/scanner-agent/agent/scan_coverage.py

Central / Durable per-IP progress to the UI

The agent submits a complete snapshot for all assigned IPs, including hosts still waiting for a worker slot.



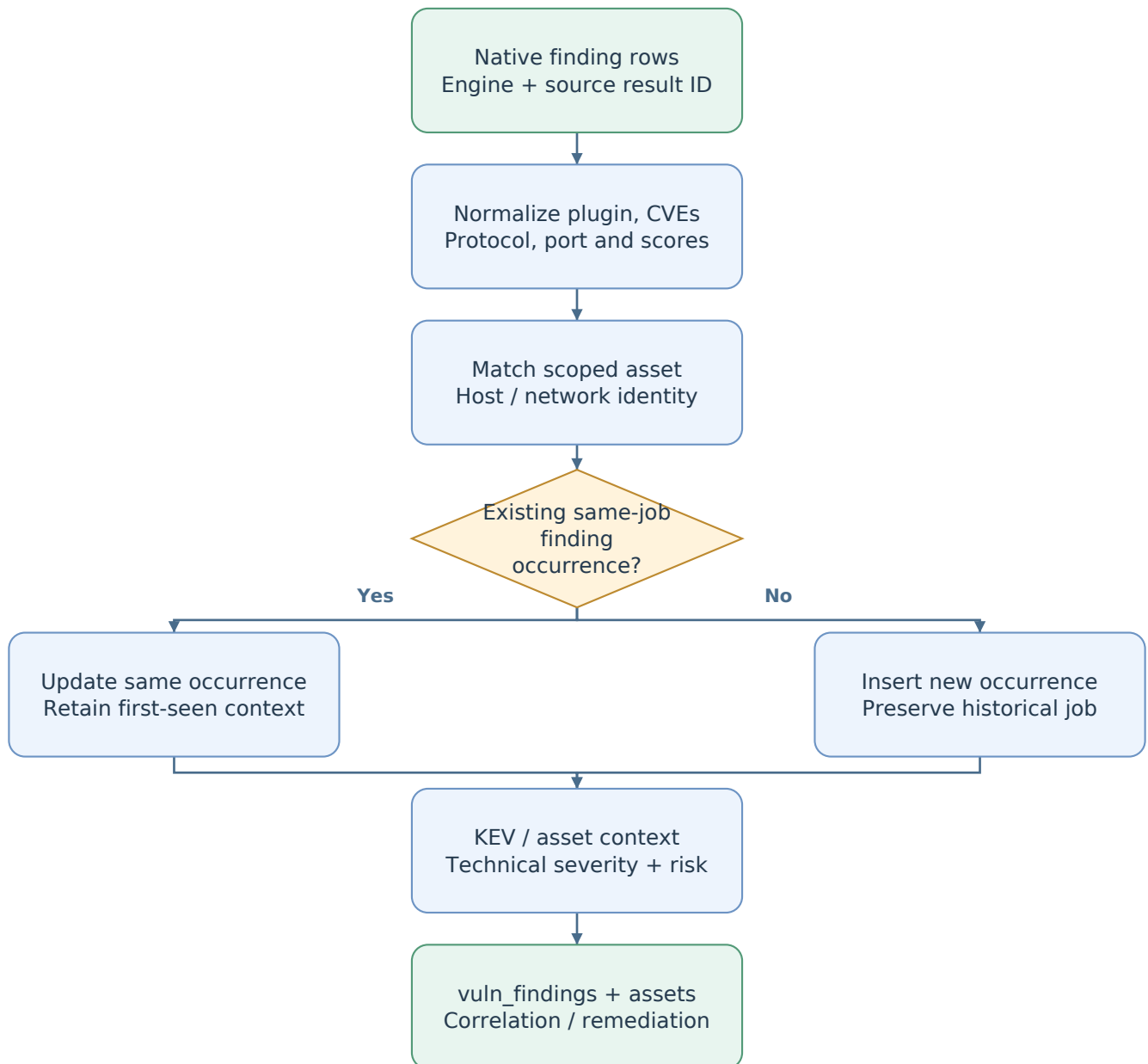
Displayed states Waiting = 0%; running stays below 100%. The per-job sequence is restored on resume; stale or duplicate events are rejected before a different task ID can reopen a host. Same-task percent cannot decrease.

Ownership / races Non-owner updates are ignored. Completed jobs cannot reopen from late polling. Row locks prevent simultaneous progress and result ingestion from overwriting each other's JSON.

Implementation: agent/target_progress.py; backend/app/services/scanner_agent_jobs.py; scan_orchestrator.py; frontend/src/pages/vuln/ScanJobsPage.tsx

Central / Normalize, deduplicate and retain findings

Native scanner rows remain linked to the assessment that produced them. Retries within a job do not reassign historic findings to a later scan.



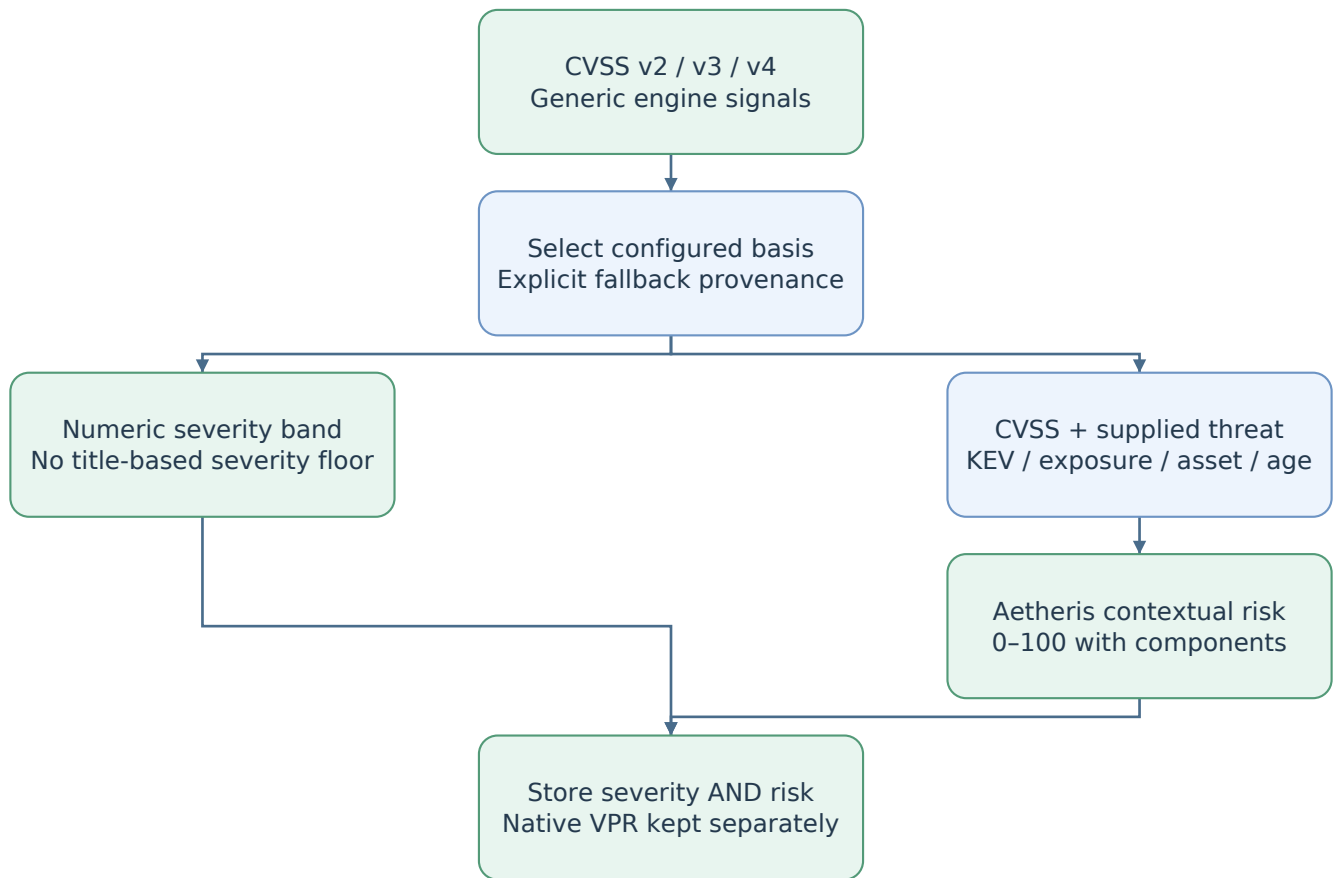
Identity Deduplication uses scan job, asset, source result ID, plugin, CVE, port and protocol. Do not merge separate host/port occurrences merely because their titles match.

Score provenance Retain version-specific CVSS candidates, chosen source/basis, scanner labels, native VPR/EPSS if supplied and QoD. Auxiliary engines may map labels to numeric scores; those mappings are not native CVSS vectors.

Implementation: backend/app/services/vuln_finding_ingest.py; aetheris_severity.py; vuln_kev_feed.py

Scoring / Technical severity and contextual priority

The configured CVSS version selects technical severity. A separate local contextual risk number supports prioritization.



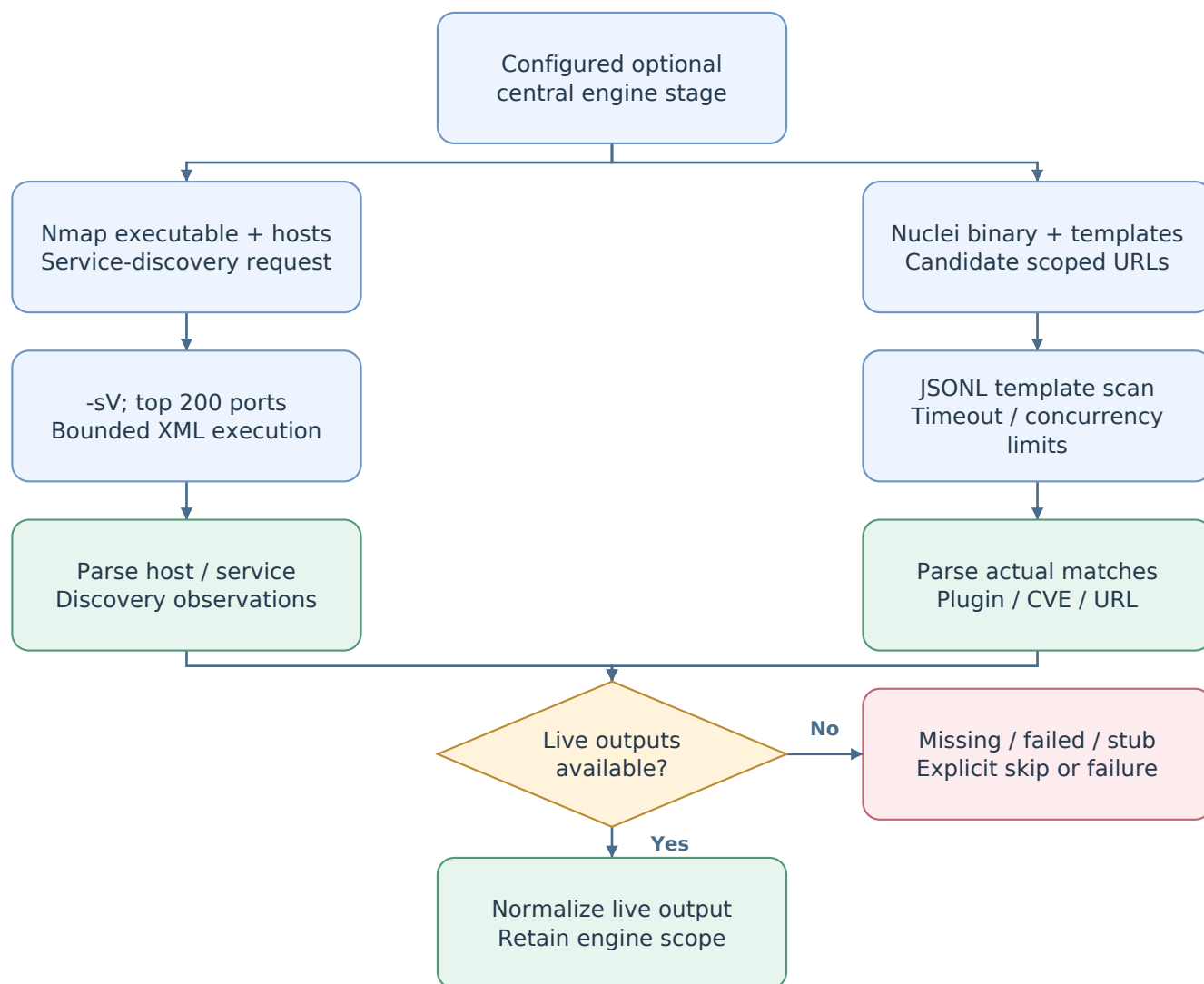
Signal	Classification / contribution
Technical severity	Default CVSS v3; v3/v4 Critical ≥ 9.0 , High ≥ 7.0 , Medium ≥ 4.0 . CVSS v2 Critical =10.0.
Local risk components	Technical ≤ 25 ; supplied VPR/EPSS threat ≤ 20 ; KEV 15; exposure ≤ 15 ; impact ≤ 15 ; age ≤ 5 ; confidence ≤ 5 .
Local risk bands	Critical ≥ 80 ; High ≥ 60 ; Medium ≥ 35 ; Low > 0 ; Info =0.

VPR boundary MODEL_VERSION=nessus-aligned-weighted-1 is a custom Aetheris contextual model. It is not Tenable VPR. Absent threat data contributes zero; CVSS does not fabricate threat probability.

Implementation: backend/app/services/aetheris_severity.py; vuln_risk.py; Tenable risk metric documentation

Central optional / Nmap and Nuclei execution

These central engines require their own executable and scoped targets. Their outputs are not a replacement for the laptop's full OpenVAS assessment.



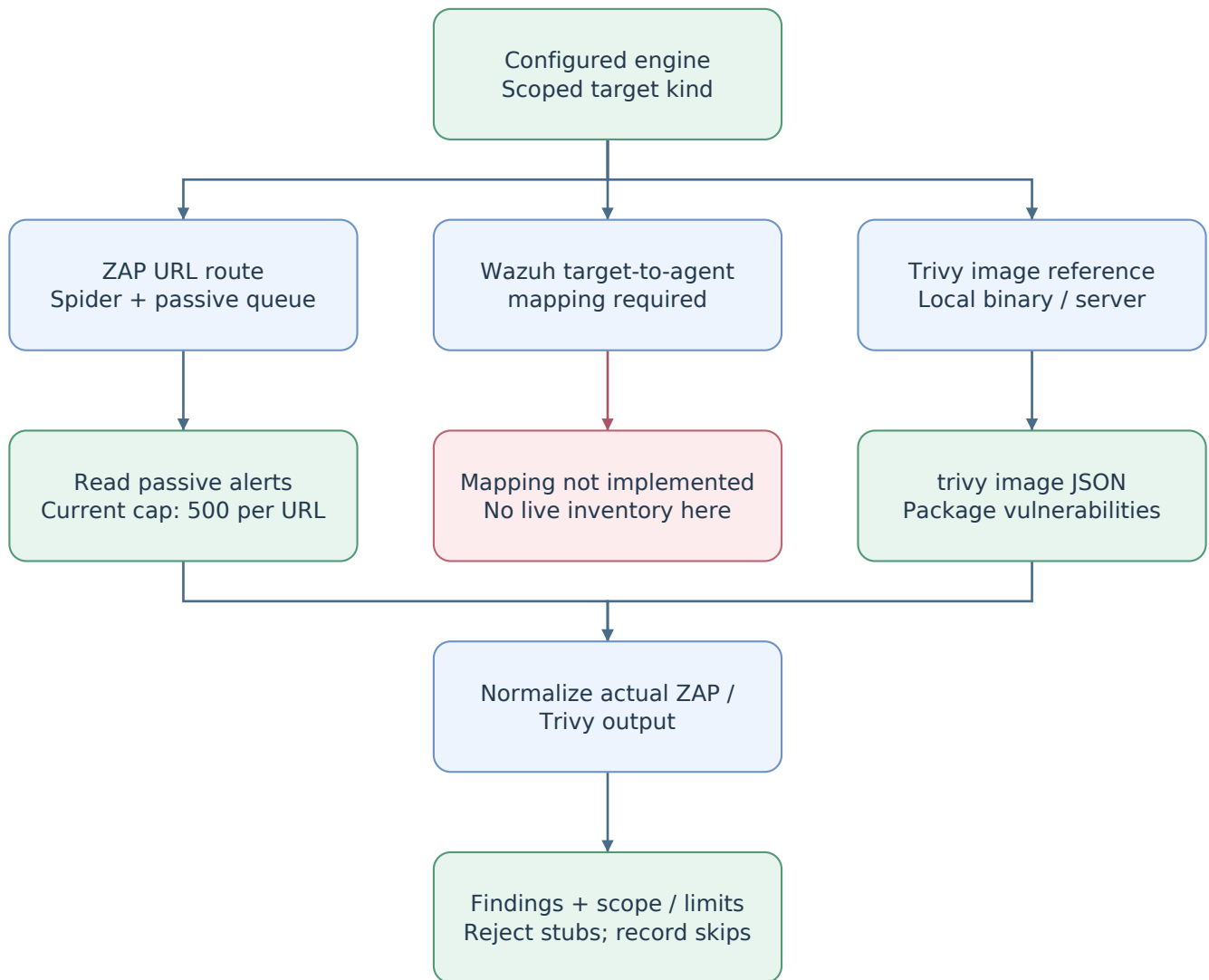
Nmap scope The implemented auxiliary scan uses top-200 discovery/service ports, not all 65,535 TCP ports. Record service observations as that bounded scope.

Nuclei scope Configured templates check matching known conditions at attempted URLs. No matches do not prove the entire host or every application vulnerability was assessed.

Implementation: backend/app/services/multi_scanner_clients.py (NmapScanner, NucleiScanner); scan_orchestrator.py

Central optional / ZAP, Trivy and Wazuh routes

The actual central implementation has distinct capabilities and limitations. Unsupported routes are not drawn as a successful live scan.



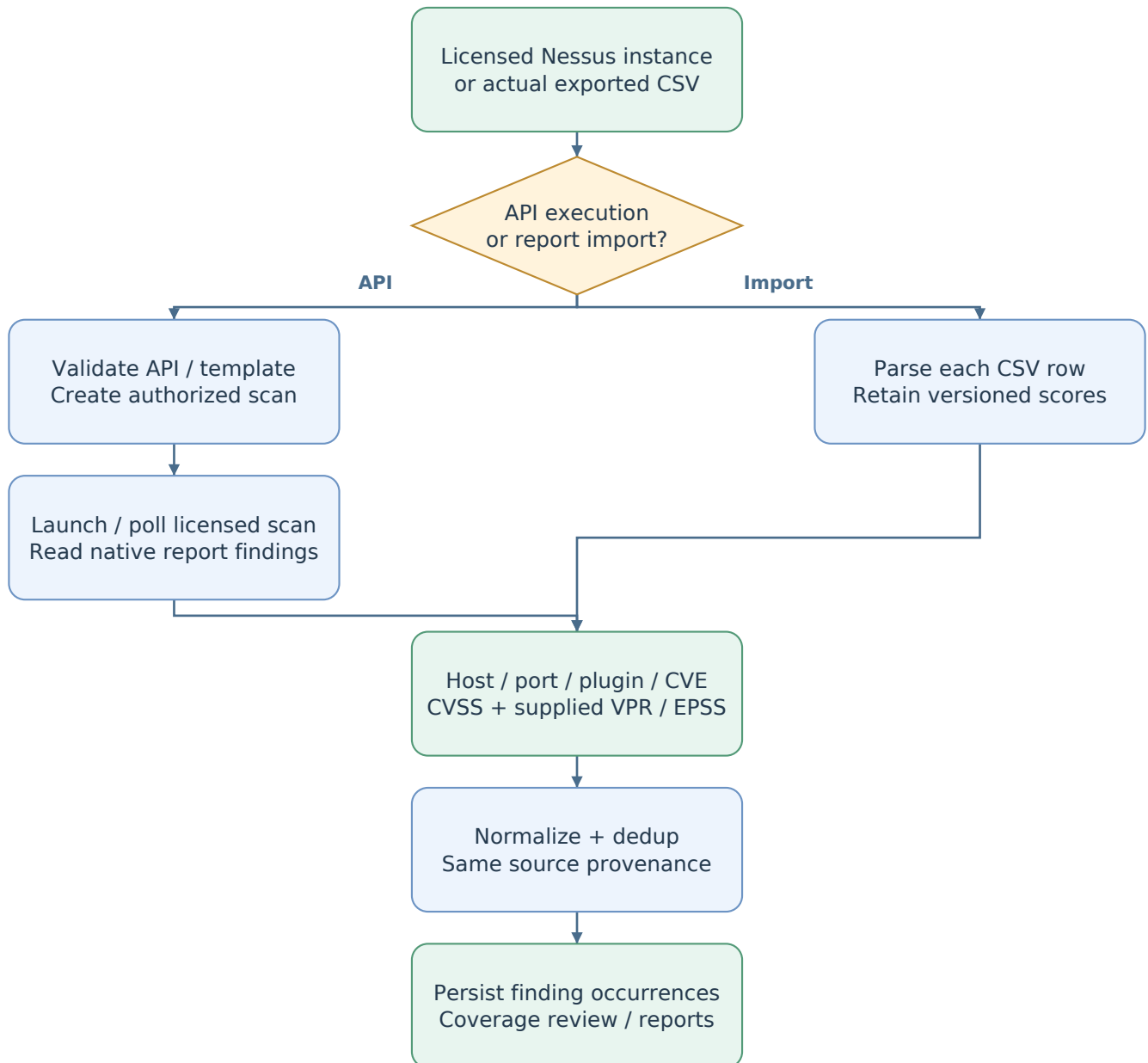
ZAP limitations The current adapter performs spidering and passive alert retrieval; it does not implement an active-scan flow here. Its 500-alert cap is a coverage limitation to review before relying on a large-site assessment.

Trivy / Wazuh Trivy scans container image references and skips IP-only targets. Wazuh currently reports that enrolled-agent target mapping is missing; a configured API URL alone does not implement live endpoint inventory.

Implementation: backend/app/services/multi_scanner_clients.py (ZapScanner, TrivyScanner, WazuhScanner)

Central optional / Licensed Nessus and native imports

The product can use a separately configured licensed Nessus API or an actual Nessus CSV export. The Laptop Scanner itself is OpenVAS.



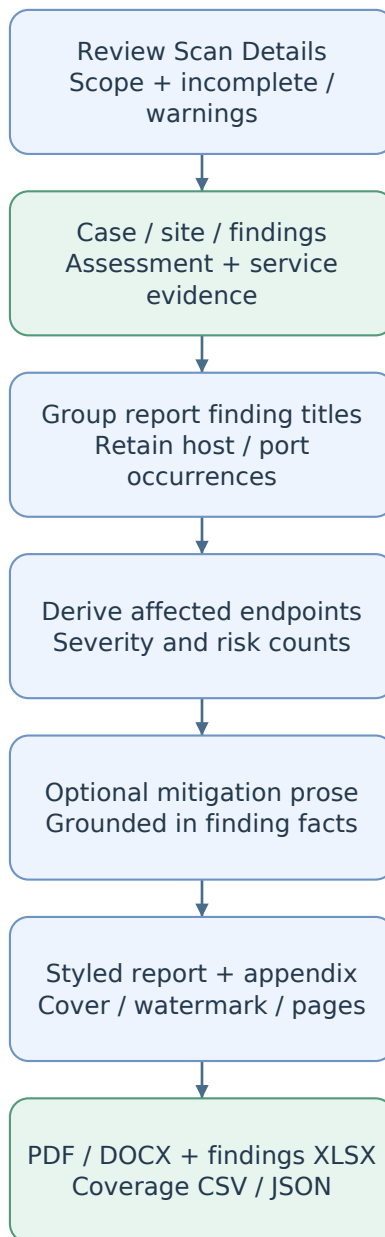
License boundary OpenVAS is not presented as the licensed Nessus engine. Similar technical severity bands do not reproduce proprietary Nessus plugins, coverage or VPR.

Import retention Retain every exported host/protocol/port row and all supplied score signals. A native VPR score remains a supplied signal, distinct from the local contextual risk number.

Implementation: backend/app/services/nessus_client.py; nessus_report.py; nessus_sync.py

Scanner report / Coverage review to gap assessment

Reports load current case/site data, persisted findings, assessment status and durable service-family evidence. The client PDF/DOCX now includes a coverage appendix.



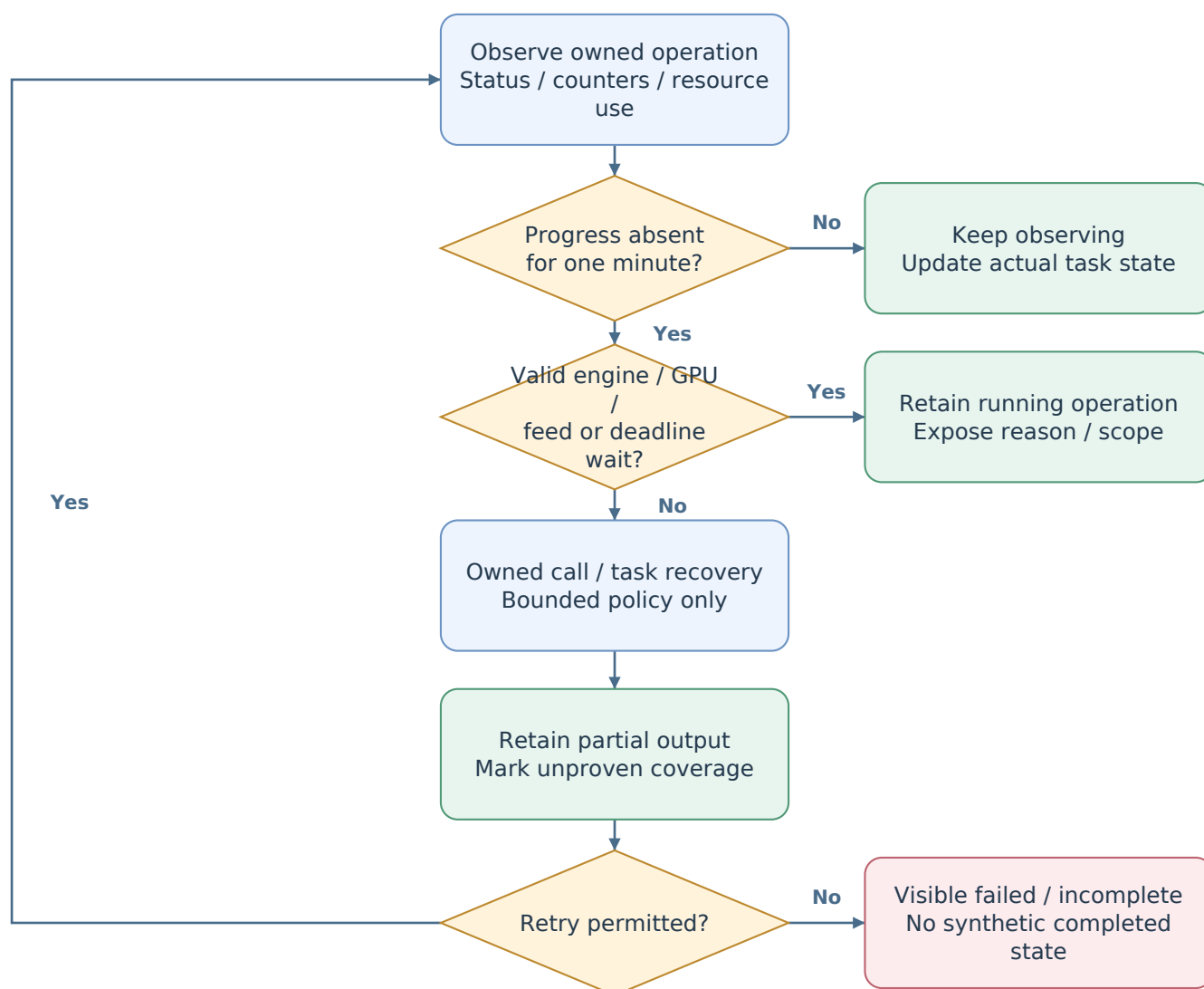
Coverage boundary The appendix distinguishes native engine assessment from per-rule coverage and lists missing/unsupported evidence. Export remains available for incomplete scans; it does not certify all 52 families passed. The companion CSV has all families per host.

Exports Solution CSV/XLSX keeps the approved eight-column layout. Separate service-coverage CSV/JSON includes family states and source result IDs; JSON retains task scope, feed/preferences and hashes. Case merges retain each job's coverage.

Implementation: backend/app/services/gap_assessment_report.py; gap_report_template.py; gap_report_llm.py; vuln_report_export.py

Monitoring / Performance without false completion

The forensic progressAgent rule and the portable scanner's task loop are different runtime boundaries. Both need valid waits to remain visible.



Implemented performance changes Canonical packaging avoids stale agents; complete snapshots remove log-only status; adaptive five/six IP scheduling fills slots; bounded polling and paged report reads preserve results. Critical pressure pauses new work.

Control boundary progressAgent is the sole outside coordinator for forensic serial/report jobs. The laptop scanner polls its owned OpenVAS tasks internally; this release does not claim that one central forensic agent can control a disconnected laptop's processes.

Implementation: backend/app/services/progress_agent.py; laptop-scanner/scanner-agent/agent/main.py; capacity.py

Release 1.5.3 / Changes and verification

The supplied standalone scanner and the canonical embedded scanner are synchronized. The central packager now preserves the modern source.

Change	Result / validation
Canonical packaging	Stop deleting modern laptop code or copying the older repo-root agent; exclude secret/runtime/cache/log files.
Progress	Complete IP snapshots; waiting/running/failed/skipped/incomplete states; percent clamping; resume preserves assessed hosts.
Readiness	Only authenticated scanner heartbeat updates; API/UI launch fence; explicit engine readiness fields.
Coverage flags	Persisted evidence, orchestration and API agree on partial/clean/warnings even when the task says Done.
Port / capacity defaults	Full TCP + priority UDP; Full optimize_test=no; floor of five workers; pressure pauses new starts.
Guide policy / evidence	52 registered service families + IP forwarding; custom scope, UDP transports, local credential references, family status, policy audit, coverage exports and report appendix.
Severity / priority	Version-aware CVSS; retain numeric signals; no title-based floors or fabricated threat probability; contextual risk separate from VPR.
Verification completed	102 standalone tests; 107 embedded agent tests; 175 focused central/report tests; PostgreSQL/WASM behavior; Vuln + Unified builds; Python 3.11 syntax and changed PowerShell grammar.
Live verification pending	Windows PowerShell execution, actual Docker/Greenbone scan timing/coverage, and live phone/GPU acceptance were not executed on target hardware.

Test semantics Regression tests include a seven-IP scheduling simulation and a finished task with degraded port scanning. Database behavior checks use actual PostgreSQL-compatible execution, not only a mocked query string.

Remaining optional-engine limits ZAP pagination beyond its current 500-alert cap and Wazuh target-to-agent mapping are documented limitations of separate central adapters; they are not installed laptop-agent processes.

Implementation: SCANNER-INTEGRATION-V153-RELEASE.md; SCANNER-INTEGRATION-V153-MANIFEST.json

Deployment / Apply the Laptop and central fixes

Update both source folders before testing live progress. Preserve existing local .env files and runtime tokens; do not reuse a packaged scanner identity.

Host / step	Command / expected result
Central source	Extract the updated polaron-disk-mobile-serial.zip into the existing project, preserving local environment and data.
Central vulnerability service	From the project root: powershell -ExecutionPolicy Bypass -File .\scripts\Apply-Scanner-Integration.ps1
Custom central configuration	Use -ProjectName and -ComposeOverrides for the deployment's existing Compose name/overlays. Use -IncludeGateway only when that gateway must rebuild shared UI/API routing.
Laptop source	Extract Aetheris-Laptop-Scanner-v1.5.3.zip into the existing laptop-scanner folder. Existing .env and scanner-agent/runtime are retained.
Laptop agent update	From that folder: powershell -ExecutionPolicy Bypass -File .\Apply-Scanner-v1.5.3.ps1
Effective scan coverage	The update retains .env. Check output: PORT_PROFILE=full and UDP_PROFILE=priority for full TCP/default priority UDP. Explicit older profile overrides still apply.
Fresh installation	Use Start-Laptop.cmd to configure the scanner and bind it to the authorized central account.
Live acceptance	Wait for real Greenbone readiness. Launch known authorized targets; verify waiting/running IPs, retained coverage, native findings, and API/UI flags. Read Scan Details before final reports.

What the scripts do Rebuild/recreate the relevant application/agent containers, validate Compose/imports and run the agent test suite. They preserve database/feed volumes and do not remove real local credentials.

Validation limit Changed PowerShell syntax was parsed successfully here. Native Windows/Docker execution remains a required target-host check.

Implementation: scripts/Apply-Scanner-Integration.ps1; laptop-scanner/Apply-Scanner-v1.5.3.ps1

Implementation map and primary references

Project paths trace every diagram to the supplied implementation. External references support protocol and metric definitions.

Area	Source location / primary documentation
Agent lifecycle / scheduling	laptop-scanner/scanner-agent/agent/main.py; capacity.py; target_progress.py; bootstrap_control_plane.py
GMP / native report	laptop-scanner/scanner-agent/agent/gmp_local.py; scan_coverage.py; port_discovery.py
Central API / state	backend/app/routers/scanner_agent.py; vuln.py; services/scanner_agent_jobs.py; scan_orchestrator.py
Scores / findings	backend/app/services/aetheris_severity.py; vuln_risk.py; vuln_finding_ingest.py; nessus_report.py
Other engines / reports	backend/app/services/multi_scanner_clients.py; gap_assessment_report.py; gap_report_template.py; vuln_report_export.py
Greenbone GMP	https://docs.greenbone.net/API/GMP/gmp-22.7.html
Tenable severity basis	https://docs.tenable.com/nessus/Content/ConfigureIndividualScanSeverity.htm
Tenable risk metrics	https://docs.tenable.com/nessus/Content/RiskMetrics.htm
FIRST CVSS v3.1	https://www.first.org/cvss/v3-1/specification-document

OpenDocument copy The companion .odt contains editable text/tables and embedded flow diagrams. The PDF is the inspected fixed-layout reference; the embedded diagram nodes are not individually editable shapes.

Reproducible documents docs/process-flows contains the structured diagram data and generation script in the updated central project. Use ReportLab and PyMuPDF to regenerate the PDFs and ODT.